

Number System

09 June 2026 05:08

How are numeric data items actually stored in the computer memory?

How much space(memory locations) is allocated for each type of data?

- byte, short, int, long, float, double, char, boolean.

How are characters and strings are stored in memory?

Number System:: The basics

We are accustomed to using the so called **decimal number system**.

- Ten digits:: 0,1,2,3,4,5,6,7,8,9
- Every digit position has a weight which is a power of 10.
- Base or radix is 10

Example:

$$234 = 2 * 10^2 + 3 * 10^1 + 4 * 10^0$$

$$250.67 = 2 * 10^2 + 5 * 10^1 + 0 * 10^0 + 6 * 10^{-1} + 7 * 10^{-2}$$

Binary Number System

Two digits:

- 0 and 1
- Every digit position has a weight which is a power of 2.
- Base or radix is 2

Example:

$$110 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0$$

$$110.01 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2}$$

Binary to decimal conversion

Each digit position of a binary number has a weight.

- Some power of two

A binary number:

$$B = b_{n-1}b_{n-2}...b_1b_0b_{-1}b_{-2}...b_{-m}$$

Corresponding value in decimal:

$$D = \sum_{i=-m}^{n-1} b_i \cdot 2^i$$

Examples:

$$\begin{aligned} 543210 \\ 101011_2 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 32 + 0 + 8 + 0 + 2 + 1 \\ &= 43_{10} \end{aligned}$$

$$= 43_{10}$$

$$\begin{aligned} 0.0101_2 &= 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} \\ &= 0.2500 + 0.0625 \\ &= 0.3125_{10} \end{aligned}$$

$$\begin{aligned} 101.11 &= 5 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 5 + 0.50 + 0.25 = 5.75_{10} \end{aligned}$$

Decimal-to-Binary Conversion

Consider the integer and fractional parts separately

For integer part,

- Repeatedly divide the given number by two and go on accumulating the remainder until the number becomes zero.
- Arrange the remainders in reverse order.

For the fractional part:

- Repeatedly multiply the given fraction by two.
 - Accumulate the integer part (0, 1).
 - If the integer part is one, chop it off.
- Arrange the integer part in order they are obtained.

Examples:

$$(239.25)_{10} = (?)_2$$

2	239	
2	119	1
2	59	1
2	29	1
2	14	1
2	7	0
2	3	1
2	1	1
	0	1

365

$$1110110101 = 949_{10}$$

$$0.25 \times 2$$

$$\begin{array}{l} 0 \\ 0 \\ 0 \end{array} \quad \begin{array}{l} 50 \times 2 \\ 00 \end{array}$$

$$\begin{array}{ccccccc} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 4 & & & 4 & & 4 & \end{array}$$

$$(239)_{10} = (11101111)_2$$

$$239 + 0.25 = 11101111 + .01$$

$$(239.25)_{10} = (11101111.01)_2$$

$$(1101110)_2 = 110$$

$$\rightarrow 1101_{10} = (44d)_{16}$$

2 ⁹	2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
1024	512	256	128	64	32	16	8	4	2

1023
0 to 511

1088

Python code for conversion of numbers

```

bs = '1101'
print(int(bs,2)) ← binary to int
13
print(oct(13)) ← int to octal
0o15
print(hex(13)) ← int to hex
0xd
print(bin(13)) ← int to binary
0b1101

```

Java

```

public class Coverersion
{
    public static void convertIntegerToBinary(int v){
        System.out.print("\nBinary equivalent of " + v + " is: " +
        Integer.toBinaryString(v));
        System.out.print("\n?: "+Integer.toUnsignedString(110, 2)); ← int is converted to binary (1101110)2
        System.out.print("\n ??: "+Integer.toString(1101, 16));
    }
}

```

Methods available in Java for conversion:

```

String toBinaryString(int)
String toHexString(int)
String toOctalString(int)
String toString(int)
String toString(int, int)
long toUnsignedLong(int)
String toUnsignedString(int)
String toUnsignedString(int, int)

```

Integer.to

Hexadecimal Number System

A compact way of representing binary numbers
16 different symbols(radix = 16)

1b = 2⁴ ← bits

0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9

0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	A (10)
1	0	1	1	B (11)
1	1	0	0	C (12)
1	1	0	1	D (13)
1	1	1	0	E (14)
1	1	1	1	F (15)

Binary to Hexadecimal Conversion

For the integer part:

- Scan the binary number from right to left.
- Translate each group of four bits into the corresponding hexadecimal digit.
 - Add leading zeros if necessary.

For the fractional part:

- Scan the binary number from left to right.
- Translate each group of four bits into the corresponding hexadecimal digit.
 - Add trailing zeros if necessary.

For example:-

$$\begin{array}{ccccccc} \overleftarrow{1010} & \overleftarrow{1110} & \overleftarrow{1101} & . & \overrightarrow{0110} & \overrightarrow{1111} & \overrightarrow{1111} \\ 2 & e & d & & 6 & 7 & 7 \end{array} \quad {}_2 = (2ed.67)_{16}$$

$$(0.\overrightarrow{1000} \overrightarrow{0100})_2 = (0.84)_{16}$$

8	4	2	1
1	1	0	1
1	1	1	0
1	0	0	0
0	1	0	1
1	0	1	0
	1	1	
1	1	0	1
0	0	1	1
0	0	1	0

Hexadecimal to binary Conversion

Translate every hexadecimal digit into its 4 bit binary equivalent.

$$(3A5)_{16} = (1110100101)_2$$

$$(12.3D)_{16} = (10010.0011101)_2$$